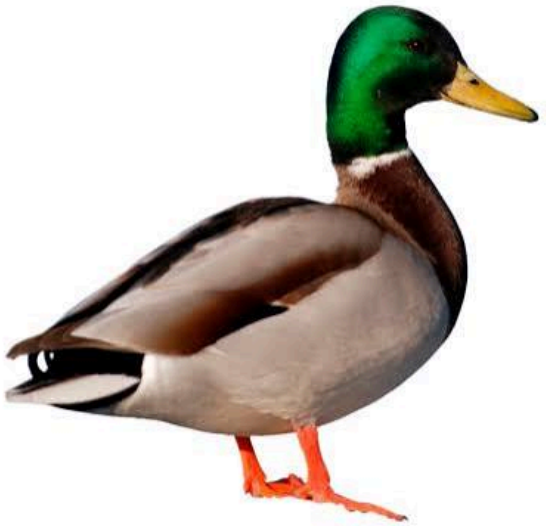
Ciência de Dados Aplicada ao Direito I

Aula 19

Classes e bibliotecas



Prof. Bruno da Cunha de Oliveira - 2026/2

Várias partes numa lista

Duas partes numa lista: o que aparece na tela?

```
class Parte:
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade
autora = Parte("Ana Lima", 66)
reu = Parte("Rui Souza", 41)
partes = [autora, reu]
for parte in partes:
    print(parte.idade)
```

A lista de partes, passo a passo

Animação no navegador: `animacao-lista-de-objetos.html`, cenário `percorrer`.

O for entrega um objeto por volta

RESPOSTA

```
66
41
```

A lista guarda os dois objetos. A cada volta, `parte` é um deles, e `parte.idade` lê o atributo desse objeto.

Escrevam o for que mostra quem tem prioridade

```
class Parte:
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade
    def tem_prioridade(self):
        return self.idade >= 60
autora = Parte("Ana Lima", 66)
reu = Parte("Rui Souza", 41)
partes = [autora, reu]
```

O método de cada parte, passo a passo

Animação no navegador: `animacao-lista-de-objetos.html`, cenário `metodo`.

Um método, dois objetos

RESPOSTA

```
for parte in partes:
    if parte.tem_prioridade():
        print(parte.nome)
```

```
Ana Lima
```

O método é o mesmo nas duas voltas, e o objeto muda. Só a autora aparece: 41 anos não chegam a 60.

Um método sem self

```
class Parte:
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade
    def tem_prioridade():
        return self.idade >= 60

autora = Parte("Ana Lima", 66)
print(autora.tem_prioridade())
```

Tirei o `self` do método. O que aparece na tela?

O objeto antes do ponto é o self

RESPOSTA

```
TypeError: Parte.tem_prioridade() takes 0 positional
arguments but 1 was given
```

Ninguém escreveu argumento entre os parênteses. Quem passou foi o Python: em `autora.tem_prioridade()`, ele entrega o objeto antes do ponto como primeiro argumento do método, e o método não tinha o parâmetro `self` para recebê-lo.

Da lista à base inteira

Do processo à base inteira

O `for` responde uma parte de cada vez. As decisões chegam como uma lista de dicionários, e as perguntas são sobre a base inteira: quantas decisões, quantas procedentes.

```
decisoes = [{"classe": "Cível", "resultado": "improcedente"},
            {"classe": "Juizado", "resultado": "procedente"},
            {"classe": "Cível", "resultado": "procedente"}]
```

Falta um tipo que guarde a base e saiba responder.

Ler a classe Tabela

```
class Tabela:
    def __init__(self, linhas):
        self.linhas = linhas
    def contar(self, campo, valor):
        achadas = []
        for linha in self.linhas:
            if linha[campo] == valor:
                achadas.append(linha)
        return len(achadas)
```

O que um objeto `Tabela` guarda? O que `contar` devolve: uma lista ou um número?

Um atributo e um método

RESPOSTA

Guarda um atributo, `linhas`, com a lista de dicionários que chegou na criação.

`contar` é um método: percorre `self.linhas`, separa as que têm aquele valor naquele campo e devolve um número, o tamanho da lista separada.

```
tabela = Tabela(decisoes)
print(tabela.contar("resultado", "procedente"))
```

```
2
```

A Tabela e o DataFrame, uma classe que alguém publicou

COM A TABELA

```
tabela = Tabela(decisoes)
len(tabela.linhas)
tabela.contar("resultado", "procedente")
```

COM O PANDAS

```
import pandas as pd
df = pd.DataFrame(decisoes)
len(df)
df["resultado"].value_counts()
```

O tamanho, nos dois lados

Animação no navegador: `animacao-tabela-dataframe.html`, cenário `tamanho`.

Contar com o DataFrame

```
import pandas as pd

decisoes = [{"resultado": "improcedente"},
            {"resultado": "procedente"},
            {"resultado": "procedente"}]
df = pd.DataFrame(decisoes)
print(df["resultado"].value_counts())
```

`value_counts()` conta cada valor da coluna. O que aparece na tela?

Contar, nos dois lados

Animação no navegador: `animacao-tabela-dataframe.html`, cenário `contar`.

O método que conta todos os valores

RESPOSTA

```
resultado
procedente      2
improcedente    1
Name: count, dtype: int64
```

A `Tabela` conta um valor por vez. O `DataFrame` conta todos de uma vez, e é esse método que aparece no programa do juscraper: `dados["assunto"].value_counts()`.

Biblioteca tem versão

O mesmo programa, três versões do pandas

Quem escreveu o `DataFrame` continua mudando a classe.

```
import pandas as pd

df = pd.DataFrame([{"resultado": "procedente"}])
nova = pd.DataFrame([{"resultado": "improcedente"}])
df = df.append(nova)
print(len(df))
```

O programa roda no pandas 1.3.5, de 2021, no 1.5.3, de 2023, e no 2.0.0, do mesmo ano. O que aparece em cada um?

Funciona, avisa, quebra

RESPOSTA

pandas	Resultado	Trecho do que aparece
1.3.5	funciona	2
1.5.3	avisa	`FutureWarning: The frame.append method is deprecated ...` e 2
2.0.0	quebra	`AttributeError: 'DataFrame' object has no attribute 'append'`

Depreciação e mudança que quebra

Nome	O que é
depreciação	o aviso de que um método vai sair, dado enquanto ele ainda funciona
mudança que quebra	a versão em que ele sai, e o programa antigo para de rodar
versão maior	o primeiro número da versão: do 1.5.3 para o 2.0.0, é nela que costuma vir a mudança que quebra

O pandas que o juscraper aceita

juscraper 0.4.0, dependências declaradas no PyPI. Trecho.

```
pandas<3.0.0,>=2.0.0
requests>=2.32.3
```

Quem escreveu o juscraper testou com o pandas 2 e travou abaixo do 3. Por isso o `pip` instalou um pandas 2 no computador de vocês, e não o mais novo.

E na segunda linha veio junto outra biblioteca: o `requests`. É com ela que o juscraper pede as decisões ao tribunal.

O pedido a um site

Onde pedir: o Tribunal Fictício

Um site do curso que imita um tribunal: tudo nele é inventado, e o formato é o de um tribunal de verdade. Pedido com um número de processo, ele responde com os dados do processo.

```
tribunal-ficticio-cdad.fly.dev/api/
processo?numero=0000101-95.2026.8.99.0001
```

O que vem depois do `?` são os parâmetros do pedido. No `requests`, eles vão separados, em `params=`.

Um pedido ao Tribunal Fictício: o que aparece na tela?

```python
import requests
url = "https://tribunal-ficticio-cdad.fly.dev/api/processo"
numero = "0000101-95.2026.8.99.0001"
resposta = requests.get(url, params={"numero": numero})
print(resposta.status_code)
dados = resposta.json()
if resposta.status_code == 200:
    print(dados["classe"]["nome"])
else:
    print(dados["erro"])
```

Um pedido e a resposta, passo a passo

Animação no navegador: `animacao-http.html`, cenário `achou`.

Pedido, resposta, atributo e método

RESPOSTA

```
200
Procedimento Comum Cível
```

Rodem no computador de vocês e confiram.

`requests.get` devolve um objeto: a resposta. `resposta.status_code` é um atributo, o código que o servidor mandou, e 200 quer dizer que deu certo. `resposta.json()` é um método: transforma o texto da resposta num dicionário.

Um número fora do cadastro: o que aparece na tela?

```
import requests
url = "https://tribunal-ficticio-cdad.fly.dev/api/processo"
numero = "0000999-11.2026.8.99.0001"
resposta = requests.get(url, params={"numero": numero})
print(resposta.status_code)
dados = resposta.json()
if resposta.status_code == 200:
    print(dados["classe"]["nome"])
else:
    print(dados["erro"])
```

O pedido que não acha, passo a passo

Animação no navegador: `animacao-http.html`, cenário `nao-achou`.

O 404 também é resposta

RESPOSTA

```
404
processo não encontrado
```

O tribunal não tem esse processo, e mesmo assim o servidor respondeu: com o código 404, e o corpo explica o motivo. Sem o `if`, o programa pediria `dados["classe"]` a um dicionário que não tem essa chave e pararia em `KeyError`.

O pedido dentro do juscraper

O cjpg por dentro

juscraper, `cjpg_download.py`. Trecho, com cortes.

```
query = {
    'dadosConsulta.pesquisaLivre': pesquisa,
    ...
    'dadosConsulta.ordenacao': 'DESC'
}

return session.get(f"{u_base}cjpg/pesquisar.do", params=query)
```

Quando vocês rodaram `tjsp.cjpg("golpe do pix")`, qual linha pediu as decisões ao tribunal? Onde entrou o "golpe do pix"?

O mesmo get, ao tribunal de verdade

RESPOSTA

A última linha: `session.get(...)` é o mesmo `get` do `requests`, chamado num objeto que o juscraper cria uma vez e reusa em todos os pedidos. O endereço é o da consulta de julgados do TJSP, e `params=query` leva a pesquisa, como o `params` do Tribunal Fictício.

O "golpe do pix" entrou em `'dadosConsulta.pesquisaLivre': pesquisa`. O resto do `cjpg` lê a página que voltou e monta o `DataFrame`.

Uma raspagem de vocês

Atividade: uma raspagem com o juscraper

#	No seu clone de `praticas-github-cdad-2026-2`
1	Em casa, uma coleta com o juscraper sobre um tema de vocês, salva em CSV; na rede do Insper, 55 pedidos ao mesmo tribunal arriscam bloqueio
2	O código e o CSV na pasta `raspagens/seu-usuario/`
3	`git add`, `git commit` e `git push`, e o pull request aberto a partir do fork

Vale 0,1 ponto extra na prova final, com o pull request aberto até o início da aula de terça, 27/10.

Na próxima aula

Como tirar de uma página de tribunal só o que interessa, e como um programa conversa com um modelo de linguagem.

Até o início dela, terça, 27/10: o pull request da raspagem, e uma conta na Groq para criar a chave em sala.